
On Safe Reinforcement Learning – Algorithms, Experiments, and Theory

Arka Ian Goswami

Reza Alvandi

Abstract

We evaluate four safe reinforcement learning algorithms (PPO, PPO-Lag, CPO, and LTL-PPO) on two distinct benchmarks: a discrete hierarchical gridworld and a continuous constrained control task. In continuous settings, PPO-Lag maintains a stable performance-safety tradeoff, whereas CPO suffers from approximation errors that cause policy collapse. We also find that standard policy gradient methods are remarkably fragile to reward design in structured environments. If the reward encoding hierarchical dependencies is removed or penalized, PPO and PPO-Lag fail to bootstrap learning. LTL-PPO overcomes this by recovering the necessary inductive bias directly from the task specification. This suggests that the choice of where to encode structural knowledge (via reward shaping versus formal specification) is just as critical as the choice of the optimization algorithm.

1 Introduction

The standard reinforcement learning (RL) objective seeks to maximize the expected cumulative return, defined as $\max_{\pi \in \Pi} \mathbb{E}^{\pi} [\sum_{t=0}^{\infty} \gamma^t R_t]$. However, framing RL strictly as an unconstrained stochastic optimization problem poses significant risks; in the blind pursuit of maximizing reward, an agent may exploit the environment and engage in undesirable or unsafe behaviors [7]. In risk-sensitive scenarios, it is necessary to either account for this unpredictability through uncertainty quantification or explicitly steer the algorithm toward a safe strategy. In light of recent advancements, it is now feasible to integrate safety constraints directly into the policy optimization framework.

Concerns regarding the safety of RL algorithms are not new. Historically, approaches to safety have been highly application-specific. In robotics, safety requirements are often enforced via reward shaping or *Linear Temporal Logic* (LTL) [5]. Conversely, financial applications typically adopt static metrics (e.g., Conditional Value-at-Risk) or dynamic risk measures (e.g., entropic risk). Because there is no single, canonical risk-sensitive objective, algorithms such as Proximal Policy Optimization (PPO) [15], PPO with Lagrangian relaxation (PPO-Lag) [11], and Constrained Policy Optimization (CPO) [1] have emerged. These methods provide a generalized mechanism to account for safety directly within the optimization objective, avoiding the need to hand-craft logic formulae or domain-specific risk measures.

In this paper, we evaluate the efficacy of these policy optimization methods across two structurally distinct benchmarks: a custom hierarchical, discrete gridworld and a high-dimensional, continuous control task. By comparing these diverse algorithms, we aim to systematically assess their constraint satisfaction, sample efficiency, and overall reliability. Furthermore, we analyze how the underlying structure of the environment itself fundamentally influences the learning dynamics and success of each algorithm.

2 Theoretical Background

Due to space constraints, we provide only a high-level overview of the evaluated algorithms. For a foundational primer on general policy optimization and Lagrangian mechanics, we refer the reader to Appendices A and B, respectively. Extended algorithmic derivations and mathematical proofs are provided in Appendix C.

Preprint.

TRPO and PPO – Trust Region Policy Optimization (TRPO) maximizes a surrogate advantage objective subject to a KL-divergence constraint. This constraint acts as a preventive measure against destructively large policy updates, ensuring the new policy remains within a trusted region of the old policy:

$$\pi_{k+1} = \arg \max_{\pi \in \Pi_\theta} \hat{J}_{\pi_{\theta_k}}(\pi), \quad \text{s.t. } \bar{D}_{KL}(\pi_{\theta_k} || \pi) \leq \delta \quad (1)$$

where $\hat{J}_{\pi_{\theta_k}}(\pi) = \mathbb{E}[\frac{\pi(a|s)}{\pi_{\theta_k}(a|s)} \hat{A}_{\pi_{\theta_k}}^R(s, a)]$, and $\bar{D}_{KL}$ is the expected KL-divergence. Proximal Policy Optimization (PPO) simplifies this computationally heavy constraint by replacing it with a clipped objective. Based on the sign of the advantage, PPO directly truncates the probability ratio to mathematically penalize overly large updates:

$$\hat{J}_{\pi_{\theta_k}}^{\text{clip}}(\pi_{\theta_{k+1}}) = \mathbb{E} \left[\min \left\{ \frac{\pi_{\theta_{k+1}}(a|s)}{\pi_{\theta_k}(a|s)} \hat{A}_{\pi_{\theta_k}}^R(s, a), \text{clip} \left(\frac{\pi_{\theta_{k+1}}(a|s)}{\pi_{\theta_k}(a|s)}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{\pi_{\theta_k}}^R(s, a) \right\} \right] \quad (2)$$

CMDPs and PPO-Lag: To enforce safety, Constrained Markov Decision Processes (CMDPs) optimize the reward subject to a strict cost threshold d : $\max_{\theta} J^R(\pi_{\theta})$ such that $J^C(\pi_{\theta}) \leq d$. This constrained problem is typically solved via Lagrangian relaxation (detailed theoretically in Appendix B):

$$\min_{\lambda \geq 0} \max_{\theta} \left\{ J^R(\pi_{\theta}) - \lambda (J^C(\pi_{\theta}) - d) \right\} \quad (3)$$

PPO-Lagrangian (PPO-Lag) adapts this for policy gradients by replacing the standard advantage function in 2 with a penalized Lagrangian advantage: $\hat{A}_{\pi_{\theta_k}}^{\text{Lag}}(s, a) = \hat{A}_{\pi_{\theta_k}}^R(s, a) - \lambda_k \hat{A}_{\pi_{\theta_k}}^C(s, a)$. The Lagrange multiplier λ is updated dynamically at each step via dual ascent: $\lambda_{k+1} = \max \{0, \lambda_k + \alpha (J^C(\pi_k) - d)\}$, where α is the learning rate.

Constrained Policy Optimization (CPO): While PPO-Lag enforces safety through soft adaptive penalties, CPO attempts to enforce it strictly at every update step. CPO analytically solves a trusted-region optimization problem that strictly enforces both the KL-divergence bound and the cost constraint simultaneously. By utilizing Taylor expansions of the objective and cost functions, CPO computes a projection that guarantees near-constraint satisfaction theoretically, though in practice, it can be computationally expensive and prone to approximation errors in complex, high-dimensional environments.

3 Methodology and Results

Detailed experimental configurations and hyperparameters for both environments are provided in Appendix D.

3.1 Discrete Grid

We consider a finite grid-world Markov Decision Process (MDP) defined on a rectangular lattice $S \subset \mathbb{Z}^2$ with a discrete action space $\mathcal{A} = \{\text{UP, DOWN, LEFT, RIGHT}\}$. The agent evolves according to stochastic dynamics induced by a slip mechanism, where at each time step, the selected action $a \in \mathcal{A}$ is subject to a slip probability. The environment contains a *key* that must be collected before opening a *door*, which in turn must be opened before reaching the *goal*. The grid also contains *treasures* (placed on free tiles), impassable *walls*, and unsafe *cliffs*. Transitions into walls leave the agent in place, while transitions into cliffs incur a safety cost.

This objective implicitly enforces a strict temporal ordering: *Collect Key* $\rightarrow$ *Open Door* $\rightarrow$ *Reach Goal*. Reaching the goal prematurely yields no reward, ensuring that the satisfaction of this sequence is necessary for optimality. Episodes terminate upon a completely successful sequence, reaching a prespecified time horizon, or optionally, falling into a cliff.

3.1.1 Discrete Outputs and Discussion

The results indicate that all three methods—PPO, PPO-LAG, and LTL-PPO—achieve near-perfect performance in both the fixed (3×10^5 steps) and randomized (10^6 steps) settings, with key, door,

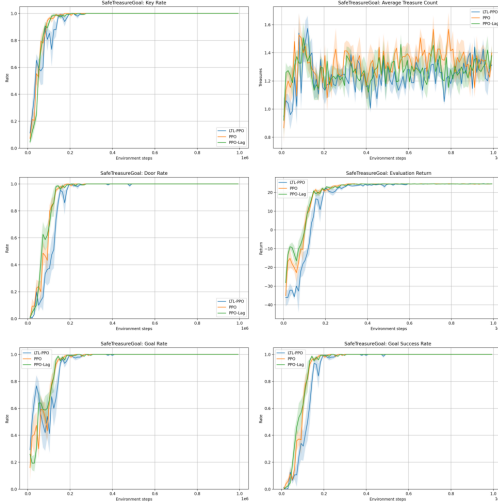


Figure 1: Algorithm sweep on the SafeTreasureGoal benchmark (random) displaying performance metrics across 5 seeds. Success rates imply a complete $K>D>G$ sequence.

goal, and overall success rates gradually converging to 1.0. In the fixed environment, convergence occurs quickly (within $\approx 1.5 \times 10^5$ steps), which is expected. While randomization increases early-stage variance and slows initial learning, it does not affect asymptotic performance, as all methods ultimately recover the same optimal behaviour (Figure 1). Across both settings, PPO-LAG typically exhibits slightly faster initial improvement, likely due to its explicit cost-sensitive updates, whereas LTL-PPO demonstrates a steep improvement in the goal rate during the initial training phase.

Crucially, the performance of these algorithms is highly sensitive to modifications in reward signals that convey the structural properties of the environment (Figure 2). Introducing an early-goal penalty (reaching the goal without the key or bypassing the door) significantly alters the learning dynamics by removing the only consistently positive signal available under random exploration. As a result, standard policy gradient methods fail to bootstrap learning and exhibit strong seed dependence: PPO succeeds in only 1/5 seeds, and PPO-LAG achieves no success at all. In contrast, LTL-PPO achieves success in 4/5 seeds. This strongly suggests that incorporating temporal logic provides a necessary inductive bias for discovering and stabilizing hierarchical behaviours under sparse and penalized reward structures.

3.2 Continuous Control

To evaluate the algorithms in a high-dimensional continuous state-action space, we benchmarked PPO, PPO-LAG, and CPO using the SafetyPointGoal1-v0 environment via the OmniSafe framework [4]. The agent must navigate a point-mass robot to randomized goal locations while avoiding dynamically placed hazards. We enforced a strict safety cost limit of $d = 25.0$. All models were trained for 10^6 timesteps (500 epochs) across 3 concurrent random seeds.

3.2.1 Continuous Outputs and Discussion

As depicted in Figure 3, standard PPO acts as an unconstrained baseline; it rapidly maximizes reward but systematically violates the safety threshold. The constrained algorithms present a fascinating contrast in reliability versus peak performance. CPO exhibits severe seed sensitivity and high variance. While it successfully achieves higher episodic returns than PPO-LAG in two of the three evaluated seeds, its strict analytical projections leave it vulnerable to approximation errors. This fragility is evident in the remaining seed, which suffers from policy collapse and remains trapped in overly conservative recovery behaviors with near-zero reward. Conversely, PPO-LAG provides a highly stable, albeit slightly more conservative, middle ground. It learns the navigation task while adaptively pulling its cost down to hover tightly around the $d = 25$ limit across all seeds. To better visualize this constraint-satisfaction trade-off, we plot the Pareto frontier (Figure 4) representing the final converged performance of each seed. The ideal policy lies in the top-left quadrant (high reward, cost ≤ 25). This visualization makes the algorithmic compromises stark: PPO is highly

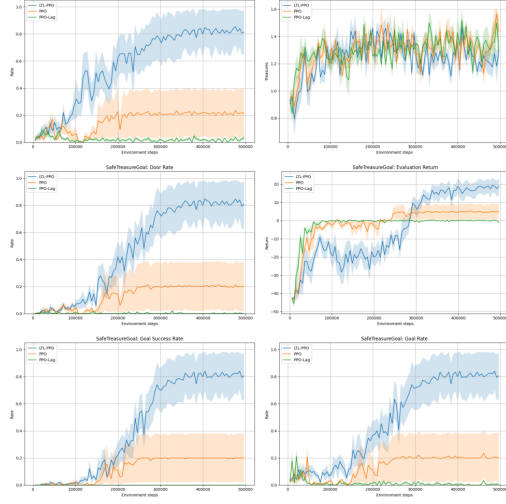


Figure 2: Algorithm sweep on the SafeTreasureGoal benchmark (early-goal penalty) displaying performance metrics across 5 seeds. Success rates imply a complete $K>D>G$ sequence.

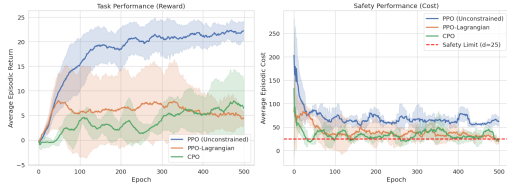


Figure 3: Task performance (Reward) and Safety performance (Cost) over 10^6 timesteps. Solid lines represent the mean over 3 random seeds. The red dashed line denotes the safety limit $d = 25$.

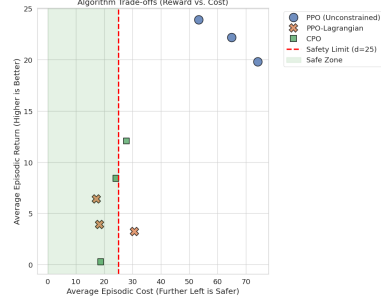


Figure 4: Pareto trade-off between final episodic return and cost. The green shaded area represents the safe constraint zone ($d \leq 25$).

performant but fundamentally unsafe (far right). CPO is capable of high performance within the safe zone, but its instability makes it an unreliable choice for safety-critical deployment. Finally, PPO-LAG successfully and reliably clusters the agent’s behavior near the safe boundary, demonstrating the robust efficacy of the Lagrangian multiplier in balancing the performance-safety trade-off in continuous control.

4 Conclusion and Future Directions

In this work, we evaluated the efficacy of constrained policy optimization across structurally diverse environments. Our experiments demonstrate that while these algorithms are capable of safety-constrained RL, their reliability heavily depends on the underlying task structure. In high-dimensional continuous control, PPO-LAG successfully balances the performance-safety trade-off, whereas strict analytic methods like CPO exhibit high variance and remain vulnerable to approximation errors and catastrophic policy collapse. Furthermore, we assessed PPO, PPO-LAG, and LTL-shaped PPO on our discrete gridworld environment. Experiments highlight a fundamental fragility in standard policy gradient methods: when hierarchical dependencies are subjected to sparse or penalized reward structures, standard methods fail to bootstrap learning. Conversely, LTL-PPO consistently succeeds. This implies that for a given environment where the goal of the problem remains the same across various reward shaping options, any hierarchical or structural information useful for learning must be encoded representationally. While our environments assumed fully observable states, real-world safety-critical systems often suffer from sensor noise and environmental occlusions. Therefore, a natural and necessary extension of this work is to formulate the constrained environments as Partially Observable Markov Decision Processes (POMDPs) [8]. Investigating how belief-state tracking and recurrent policy architectures interact with strict Lagrangian penalties or LTL specifications will be crucial for bridging the gap between simulated benchmarks and practical deployment. Additionally, we intuitively hypothesize that the adverse learning dynamics caused by the early-goal penalty could be mitigated by modeling full return distributions instead of scalar expected values [3].

Code, Contributions, and Acknowledgments

Code and Reproducibility: The complete codebase, detailed implementation notes, and the early-goal penalty impact are publicly available at <https://github.com/reza2002801/safe-rl-benchmarks>. *Note:* Due to the strict page limit, we encourage the reader to consult the appendices for extended theoretical derivations, full hyperparameter configurations, and an additional experiment omitted from the main text.

Contributions: All aspects of this project were equally contributed to by both members. Experiments were developed collaboratively and verified independently.

Acknowledgments: Both members are affiliated with the Systems and Control Group in ECE and express gratitude to Prof. Aditya Mahajan and Prof. Peter Edwin Caines for their supervision and financial support. We thank the Farama Foundation for their open-source RL benchmarks.

References

- [1] Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. Constrained policy optimization. In *International Conference on Machine Learning (ICML)*, 2017.
- [2] Alekh Agarwal, Nan Jiang, Sham M. Kakade, and Wen Sun. Reinforcement learning: Theory and algorithms, 2021. Lecture notes / monograph.
- [3] Marc G. Bellemare, Will Dabney, and Rémi Munos. *Distributional Reinforcement Learning*. MIT Press, Cambridge, MA, 2023.
- [4] OmniSafe Contributors. Omnisafe: A comprehensive safe reinforcement learning benchmark and library. <https://github.com/PKU-Alignment/omnisafe>, 2023. Accessed: 2026.
- [5] Xu Chu Ding, Stephen L. Smith, Calin Belta, and Daniela Rus. Optimal control of markov decision processes with linear temporal logic constraints. *IEEE Transactions on Automatic Control*, 59(5):1244–1257, 2014.
- [6] Jonathan Eckstein and Dimitri P. Bertsekas. On the douglas–rachford splitting method and the proximal point algorithm for maximal monotone operators. *Mathematical Programming*, 55:293–318, 1992.
- [7] Javier García and Fernando Fernández. A comprehensive survey on safe reinforcement learning. *Journal of Machine Learning Research*, 16(1):1437–1480, 2015.
- [8] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998.
- [9] Sham M. Kakade and John Langford. Approximately optimal approximate reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2002.
- [10] Neal Parikh and Stephen Boyd. *Proximal Algorithms*, volume 1 of *Foundations and Trends in Optimization*. Now Publishers Inc., 2014.
- [11] Alex Ray, Joshua Achiam, and Dario Amodei. Benchmarking safe exploration in deep reinforcement learning. *arXiv preprint arXiv:1910.01708*, 7(1):2, 2019.
- [12] R. Tyrrell Rockafellar and Roger J.-B. Wets. *Variational Analysis*, volume 317 of *Grundlehren der mathematischen Wissenschaften*. Springer, 1998.
- [13] Walter Rudin. *Functional Analysis*. McGraw-Hill, 2nd edition, 1991.
- [14] John Schulman, Sergey Levine, Philipp Moritz, Michael Jordan, and Pieter Abbeel. Trust region policy optimization. In *International Conference on Machine Learning (ICML)*, 2015.
- [15] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [16] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2nd edition, 2018.

Appendix

A Policy Optimization

Before detailing the specific algorithms, we provide a brief background on policy optimization. For further insights into generalization and the specific mechanisms of policy optimization, we refer the interested reader to [2], while [16] serves as a foundational introduction.

Policy optimization is a subclass of stochastic optimization. In PO, one seeks to resolve a *function optimization* problem $\max_{\pi \in \Pi} J(\pi)$, unlike the methods adopted in stochastic control (dynamic programming), which pursue the resolution of value optimization $\max_{a \in \mathcal{A}} Q(s, a)$. Function optimization is inherently more complex and lacks the rigorous theoretical guarantees of value-based methods, as classical fixed-point and contraction arguments no longer apply. However, in environments with continuous and high-dimensional state-action spaces, the exact computation of $\max_{a \in \mathcal{A}}$ becomes algorithmically infeasible, making PO a necessity.

PO Methods – Policy optimization methods are primarily bifurcated into two classes:

- **Stochastic policies** are described by the mapping $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$, where $\pi(a|s)$ represents a probability distribution over actions given a state. The classical policy gradient algorithm employs log-likelihood estimators (e.g., REINFORCE), which requires $\pi_\theta(a|s)$ to be differentiable with respect to the parameter θ .
- **Deterministic policies** pose a unique mathematical challenge. Here, $\pi : \mathcal{S} \rightarrow \mathcal{A}$ acts as a Dirac delta on a specific action, meaning its distribution over actions is non-differentiable. Consequently, the standard score function step, $\nabla_\theta \log \pi_\theta^D(a|s)$, is ill-posed, even though the deterministic policy itself remains differentiable with respect to θ . To resolve this, one typically resorts to *actor-critic methods* leveraging the deterministic policy gradient: $\nabla_\theta \pi_\theta^D(s) \nabla_a Q(s, a)|_{a=\pi_\theta(s)}$.

Proximal Optimization – Classical proximal optimization methods [10] enforce proximity between iterative updates to prevent overstepping (and the subsequent need to reverse gradient direction), ensure the surrogate objective is strongly convex, and improve the efficiency of splitting algorithms. In proximal policy optimization, these stabilization principles apply similarly, though strict convexity is typically not assumed. Furthermore, PO introduces a distribution mismatch between the data-generating policy and the policy being evaluated. This is resolved via *importance sampling*, utilizing the ratio $\frac{\pi_{\theta_{k+1}}(a|s)}{\pi_{\theta_k}(a|s)}$, which necessitates a stochastic policy. To prevent catastrophic evaluation inaccuracies stemming from large distributional shifts, a divergence constraint (typically a ϕ -divergence) is imposed.

TRPO is the foundational algorithm within this class of proximal policy optimization. PPO, CPO, and the other variants considered in this document are algorithmic extensions or modifications of the core TRPO framework.

B Unconstrained Optimization

This section draws upon foundational works in optimization [10, 6, 12]. First, consider the standard primal formulation:

$$\begin{aligned} & \min_{x \in \mathbb{R}^n} f(x) \\ \text{subject to} \quad & h_i(x) = 0, \quad i = 1, \dots, m, \\ & g_j(x) \leq 0, \quad j = 1, \dots, p, \end{aligned}$$

where f , h_i , and g_j are convex functions. This structure—an objective function accompanied by equality and inequality constraints—defines a standard convex optimization problem. An elegant method to integrate these constraints and produce an unconstrained objective is via *Lagrangian relaxation* [12, 6]:

$$\mathcal{L}(x, \lambda, \nu) = f(x) + \sum_{i=1}^m \lambda_i h_i(x) + \sum_{j=1}^p \nu_j g_j(x),$$

where λ and ν act as the *Lagrange multipliers* for the equality and inequality constraints, respectively. From this, we define the Lagrange dual function:

$$d(\lambda, \nu) = \inf_x \mathcal{L}(x, \lambda, \nu) \leq f(x^*),$$

which satisfies the Fenchel-Rockafellar duality conditions.

Dual Formulation

The dual formulation of the aforementioned primal problem is $\max_{\lambda, \nu \geq 0} d(\lambda, \nu)$. This is fundamentally a concave maximization problem, even if the original primal problem is not strictly convex. The relationship between the primal and dual is characterized by:

- **Weak Duality:** For any feasible x and multipliers $\lambda, \nu \geq 0$, it holds that $d(\lambda, \nu) \leq f(x)$.
- **Strong Duality:** If the primal problem is convex and satisfies a constraint qualification (such as Slater's condition), then the optimal bounds converge tightly: $\max_{\lambda, \nu \geq 0} d(\lambda, \nu) = \min_x f(x)$.

Karush–Kuhn–Tucker (KKT) Conditions

The KKT conditions are a set of necessary and sufficient properties that a primal-dual tuple (x^*, λ^*, ν^*) must satisfy to be deemed optimal. Under strong duality, a point is optimal if and only if it satisfies:

- *Primal Feasibility:* $h_i(x^*) = 0, \forall i$ and $g_j(x^*) \leq 0, \forall j$.
- *Dual Feasibility:* $\nu_j^* \geq 0, \forall j$.
- *Complementary Slackness:* $\nu_j^* g_j(x^*) = 0, \forall j$.
- *Stationarity:* $0 \in \partial f(x^*) + \sum_{i=1}^m \lambda_i^* \partial h_i(x^*) + \sum_{j=1}^p \nu_j^* \partial g_j(x^*)$.
If all functions are differentiable, this simplifies to:

$$\nabla f(x^*) + \sum_{i=1}^m \lambda_i^* \nabla h_i(x^*) + \sum_{j=1}^p \nu_j^* \nabla g_j(x^*) = 0.$$

C Theoretical Derivations and Proofs

C.1 TRPO Theorems and Proofs

Trust Region Policy Optimization or TRPO [14] proposes a method for approximating the performance of a future policy. The algorithm suggests a two-step function approximation process

- a probabilistic approximation - the surrogate
- and quadratic approximation for the probabilistically approximated surrogate - the copy of the surrogate which the algorithm actually solves

Suppose the arbitrary policy, given π has known performance $J^R(\pi)$. In policy optimization, one needs to find the performance of another policy expressed as

Theorem 1 (Performance Difference Bounds).

$$J^R(\pi') = J^R(\pi) + \mathbb{E}_{\tau \sim \pi'} \left[\sum_{t=0}^{\infty} \gamma^t A_{\pi}^R(s, a) \right] \iff J^R(\pi') = J^R(\pi) + \sum_s d_{\pi'}(s) \sum_a \pi'(a|s) A_{\pi}^R(s, a) \quad (4)$$

where $A_{\pi}^R(s, a)$ is the advantage and d_{π} is the (discounted) visitation frequency or the marginal of the occupancy measure, and τ is a trajectory $(s_0, a_0, s_1, a_1, \dots)$ distributed according to the next policy π' .

Proof of Theorem 1. Since $A_\pi^R(s, a) = \mathbb{E}_{s' \sim P(\cdot|s, a)}[r(s) + \gamma V_\pi^R(s') - V_\pi^R(s)]$,

$$\begin{aligned} \mathbb{E}_{\tau \sim \pi'} \left[\sum_{t \geq 0} \gamma^t A_\pi^R(s_t, a_t) \right] &= \mathbb{E}_{\tau \sim \pi'} \left[\sum_{t \geq 0} \gamma^t (r(s_t) + \gamma V_\pi^R(s_{t+1}) - V_\pi^R(s_t)) \right] \\ &= \mathbb{E}_{\tau \sim \pi'} \left[-V_\pi^R(s_0) + \sum_{t \geq 0} \gamma^t r(s_t) \right] \\ &= -\mathbb{E}_{s_0} [V_\pi^R(s_0)] + \mathbb{E}_{\tau \sim \pi'} \left[\sum_{t \geq 0} \gamma^t r(s_t) \right] \\ &= -J^R(\pi) + J^R(\pi'). \end{aligned}$$

Thus, $J^R(\pi') = J^R(\pi) + \mathbb{E}_{\tau \sim \pi'} [\sum_{t \geq 0} \gamma^t A_\pi^R(s_t, a_t)]$. $\square$

However, the visitation frequency $d_{\pi'}$ depends implicitly on the future distribution of states according to the next policy π' , which, in general, is difficult to presume. Trust Region Policy Optimization or TRPO is a method for locally approximating the performance of π' with respect to states distributed according to the current policy π . For this define a surrogate function.

$$\hat{J}_\pi^R(\pi') = J^R(\pi) + \sum_s d_\pi(s) \sum_a \pi'(a|s) A_\pi^R(s, a) \quad (5)$$

Evidently, the main and only difference between 4 and 5 is that the latter uses the visitation frequency with states distributed according to the current policy π . The main reason one changes the visitation frequency but keeps the $\pi'(a|s)$ is because the latter is local whereas $d_{\pi'}$ depends on π' at the trajectory level. This formulation now, guarantees the monotonic improvement of policies iff $\sum_a \pi'(a|s) A_\pi^R(s, a) \geq 0$.

Theorem 2. *For any parametrized policy π_θ differentiable with respect to the parameter θ , the surrogate is first-order analytic, that is,*

$$\hat{J}_{\pi_{\theta_0}}^R(\pi'_{\theta_0}) = J_{\pi_{\theta_0}}^R(\pi'_{\theta_0}) = J_{\pi_{\theta_0}}^R(\pi_{\theta_0}) \quad (6)$$

$$\nabla_\theta \hat{J}_{\pi_{\theta_0}}^R(\pi'_\theta) \Big|_{\theta=\theta_0} = \nabla_\theta J^R(\pi'_\theta) \Big|_{\theta=\theta_0} = \nabla_\theta J^R(\pi_\theta) \Big|_{\theta=\theta_0} \quad (7)$$

Proof. Proof follows directly from the natural policy gradient theorem. For brevity, this has been left inexplicit. $\square$

7 implies that a sufficiently small policy gradient step seeking to improve the surrogate $\hat{J}_{\pi_{\theta_{old}}}^R$ also improves the actual performance measure J^R .

Theorem 3 (CPI lower bound). *Let π_{old} be the current policy and let*

$$\pi_{new}(a|s) = (1 - \alpha)\pi_{old}(a|s) + \alpha\pi^*(a|s), \quad \alpha \in [0, 1],$$

where $\pi^* \in \pi'$. Define

$$\epsilon := \max_s \left| \sum_a \pi^*(a|s) A_{\pi_{old}}^R(s, a) \right|.$$

Then

$$J^R(\pi_{new}) \geq \hat{J}_{\pi_{old}}^R(\pi_{new}) - \frac{2\epsilon\gamma}{(1-\gamma)^2} \alpha^2.$$

Proof. Write

$$\bar{A}(s) := \sum_a \pi^*(a|s) A_{\pi_{old}}^R(s, a).$$

Since

$$\sum_a \pi_{old}(a|s) A_{\pi_{old}}^R(s, a) = 0,$$

the CPI mixture gives

$$\sum_a \pi_{new}(a|s) A_{\pi_{old}}^R(s, a) = (1 - \alpha) \sum_a \pi_{old}(a|s) A_{\pi_{old}}^R(s, a) + \alpha \sum_a \pi^*(a|s) A_{\pi_{old}}^R(s, a) = \alpha \bar{A}(s).$$

By the performance difference identity,

$$J^R(\pi_{\text{new}}) = J^R(\pi_{\text{old}}) + \sum_s d_{\pi_{\text{new}}}(s) \sum_a \pi_{\text{new}}(a|s) A_{\pi_{\text{old}}}^R(s, a),$$

and hence

$$J^R(\pi_{\text{new}}) = J^R(\pi) + \alpha \sum_s d_{\pi_{\text{new}}}(s) \bar{A}(s).$$

The surrogate freezes the visitation distribution at π_{old} , so

$$\hat{J}_{\pi_{\text{old}}}^R(\pi_{\text{new}}) = J^R(\pi_{\text{old}}) + \sum_s d_{\pi_{\text{old}}}(s) \sum_a \pi_{\text{new}}(a|s) A_{\pi_{\text{old}}}^R(s, a) = J^R(\pi_{\text{old}}) + \alpha \sum_s d_{\pi_{\text{old}}}(s) \bar{A}(s).$$

Subtracting,

$$J^R(\pi_{\text{new}}) - \hat{J}_{\pi_{\text{old}}}^R(\pi_{\text{new}}) = \alpha \sum_s (d_{\pi_{\text{new}}}(s) - d_{\pi_{\text{old}}}(s)) \bar{A}(s).$$

Since $|\bar{A}(s)| \leq \epsilon$,

$$J^R(\pi_{\text{new}}) - \hat{J}_{\pi_{\text{old}}}^R(\pi_{\text{new}}) \geq -\alpha \epsilon \sum_s |d_{\pi_{\text{new}}}(s) - d_{\pi_{\text{old}}}(s)|.$$

It remains to bound the visitation-frequency shift. Define

$$d_{\pi}(s) := \sum_{t=0}^{\infty} \gamma^t \Pr_{\pi}(s_t = s), \quad p_t^{\pi}(s) := \Pr_{\pi}(s_t = s).$$

For each state s ,

$$\sum_a |\pi_{\text{new}}(a|s) - \pi_{\text{old}}(a|s)| = \alpha \sum_a |\pi^*(a|s) - \pi_{\text{old}}(a|s)| \leq 2\alpha.$$

Let

$$P_{\pi}(s'|s) := \sum_a \pi(a|s) P(s'|s, a).$$

Then

$$\begin{aligned} \sum_{s'} |P_{\pi_{\text{new}}}(s'|s) - P_{\pi_{\text{old}}}(s'|s)| &= \sum_{s'} \left| \sum_a (\pi_{\text{new}}(a|s) - \pi_{\text{old}}(a|s)) P(s'|s, a) \right| \\ &\leq \sum_a |\pi_{\text{new}}(a|s) - \pi_{\text{old}}(a|s)| \sum_{s'} P(s'|s, a) \leq 2\alpha. \end{aligned}$$

Thus

$$\sup_s \|P_{\pi_{\text{new}}}(\cdot|s) - P_{\pi_{\text{old}}}(\cdot|s)\|_1 \leq 2\alpha.$$

Assume both policies start from the same initial distribution, so $p_0^{\pi_{\text{new}}} = p_0^{\pi_{\text{old}}}$. We now show

$$\|p_t^{\pi_{\text{new}}} - p_t^{\pi_{\text{old}}}\|_1 \leq 2t\alpha.$$

The claim is trivial for $t = 0$. For the induction step,

$$\begin{aligned} \|p_{t+1}^{\pi_{\text{new}}} - p_{t+1}^{\pi_{\text{old}}}\|_1 &= \|p_t^{\pi_{\text{new}}} P_{\pi_{\text{new}}} - p_t^{\pi_{\text{old}}} P_{\pi_{\text{old}}}\|_1 \\ &\leq \|p_t^{\pi_{\text{new}}} (P_{\pi_{\text{new}}} - P_{\pi_{\text{old}}})\|_1 + \|(p_t^{\pi_{\text{new}}} - p_t^{\pi_{\text{old}}}) P_{\pi_{\text{old}}}\|_1 \\ &\leq 2\alpha + \|p_t^{\pi_{\text{new}}} - p_t^{\pi_{\text{old}}}\|_1 \leq 2(t+1)\alpha, \end{aligned}$$

where the second term uses non-expansiveness of stochastic kernels in ℓ_1 . Therefore,

$$\|p_t^{\pi_{\text{new}}} - p_t^{\pi_{\text{old}}}\|_1 \leq 2t\alpha \quad \forall t \geq 0.$$

Consequently,

$$\begin{aligned} \sum_s |d_{\pi_{\text{new}}}(s) - d_{\pi_{\text{old}}}(s)| &= \sum_s \left| \sum_{t=0}^{\infty} \gamma^t (p_t^{\pi_{\text{new}}}(s) - p_t^{\pi_{\text{old}}}(s)) \right| \\ &\leq \sum_{t=0}^{\infty} \gamma^t \sum_s |p_t^{\pi_{\text{new}}}(s) - p_t^{\pi_{\text{old}}}(s)| \\ &= \sum_{t=0}^{\infty} \gamma^t \|p_t^{\pi_{\text{new}}} - p_t^{\pi_{\text{old}}}\|_1 \leq 2\alpha \sum_{t=0}^{\infty} t\gamma^t \\ &= \frac{2\gamma}{(1-\gamma)^2} \alpha. \end{aligned}$$

Substituting this into the earlier comparison gives

$$J^R(\pi_{\text{new}}) - \hat{J}_{\pi_{\text{old}}}^R(\pi_{\text{new}}) \geq -\frac{2\epsilon\gamma}{(1-\gamma)^2} \alpha^2.$$

Hence,

$$J^R(\pi_{\text{new}}) \geq \hat{J}_{\pi_{\text{old}}}^R(\pi_{\text{new}}) - \frac{2\epsilon\gamma}{(1-\gamma)^2} \alpha^2.$$

□

This proof has been adapted from [9]

C.2 Importance Sampling

As evident from the surrogate expression 5, the data used to evaluate the new policy is derived from the old one. To formulate a usable and well-posed expectation, TRPO proposes a marginal change of measure.

$$\mathbb{E}_{\mathbb{Q}_Y}[g(Y)] = \mathbb{E}_{\mathbb{P}_Y} \left[\frac{d\mathbb{Q}_Y}{d\mathbb{P}_Y}(Y) g(Y) \right], \quad (8)$$

$$\frac{d\mathbb{Q}_Y}{d\mathbb{P}_Y}(y) = \mathbb{E}_{\mathbb{P}} \left[\frac{d\mathbb{Q}}{d\mathbb{P}}(X) \mid T(X) = y \right]. \quad (9)$$

Here, $\mathbb{P}, \mathbb{Q}$ are probability measures on $\mathcal{X}$ with $\mathbb{Q} \ll \mathbb{P}$, $T : \mathcal{X} \rightarrow \mathcal{Y}$ is a measurable map, $Y = T(X)$ is the induced marginal variable, $\mathbb{P}_Y = T_{\#}\mathbb{P}$ and $\mathbb{Q}_Y = T_{\#}\mathbb{Q}$ are the pushforward measures on $\mathcal{Y}$, g is a measurable test function on $\mathcal{Y}$, and $\frac{d\mathbb{Q}}{d\mathbb{P}}, \frac{d\mathbb{Q}_Y}{d\mathbb{P}_Y}$ denote the *Radon-Nikodym derivatives* [13]. It is important noting that the expectation in the equation 9 must always get computed with respect to the original measure $\mathbb{P}$ on $\mathcal{X}$ and not using any of its marginals.

C.3 TRPO Objective

TRPO solves the following

$$\pi_{k+1} = \arg \max_{\pi \in \Pi_{\theta}} J^R(\pi_k), \quad \text{s.t. } D_{KL}(\pi_k || \pi_{k+1}) \leq \delta \quad (10)$$

The constraint applied ensures that large policy updates are discouraged as a preventive measure of overstepping important stationary points. The KL divergence gets computed as follows

$$\bar{D}_{KL}(\pi_k || \pi_{k+1}) = \mathbb{E}_{s \sim d_{\pi_k}} [D_{KL}(\pi_k || \pi_{k+1})] \quad (11)$$

Consequently, the new objective is

$$\max_{\theta_{k+1}} \hat{J}_{\pi_{\theta_k}}(\pi_{\theta_{k+1}}), \quad \bar{D}_{KL}(\pi_{\theta_k} || \pi_{\theta_{k+1}}) \leq \delta \quad (12)$$

where $\hat{J}_{\pi_{\theta_k}}(\pi_{\theta_{k+1}}) = \frac{\pi_{\theta_{k+1}}(a|s)}{\pi_{\theta_k}(a|s)} \hat{A}_{\pi_{\theta_k}}^R(s, a)$, where $\hat{A}_{\pi_{\theta_k}}^R(s, a)$ is an estimation of the advantage function given (s, a) . Clearly, one important note from 11 is that, computing the KL divergence to set up the constraint requires sampling Monte-Carlo rollouts and then performing an expectation over the sample KL divergences with states distributed according to the visitation frequency of the previous policy. That is, the argument of the expectation has a complex dependency on d_{π_k} , which in turn depends on Monte-Carlo returns making it a very messy affair, from the perspective of computation. Moreover, even the previously mentioned surrogate $\hat{J}_{\pi_{\theta_k}}(\pi_{\theta_{k+1}})$ is further quadratic-approximated to form the actual objective which TRPO solves

$$\max_x g_k^\top x, \quad \text{s.t. } \frac{1}{2} x^\top H_k x \leq \delta, \quad (13)$$

where

$$x := \theta_{k+1} - \theta_k, \quad g_k := \nabla_{\theta_{k+1}} \hat{J}_{\pi_{\theta_k}}(\pi_{\theta_{k+1}}) \Big|_{\theta_{k+1}=\theta_k}, \quad H_k := \nabla_{\theta_{k+1}}^2 \bar{D}_{KL}(\pi_{\theta_k} || \pi_{\theta_{k+1}}) \Big|_{\theta_{k+1}=\theta_k}. \quad (14)$$

13 along with the previous 5 implies that TRPO uses two levels of approximations. The approximations themselves are not necessarily universally erroneous and the hefty computation involved with the Hessian is avoidable by the use of conjugate gradient algorithms. Hence, the only persistent problem is an efficient way to compute the KL divergence. To address this, the suggestion is to use a Lagrangian relaxation of the KL constraint

$$\max_{\theta_{k+1}} \left\{ \hat{J}_{\pi_{\theta_k}}(\pi_{\theta_{k+1}}) - \beta \bar{D}_{KL}(\pi_{\theta_k} || \pi_{\theta_{k+1}}) \right\} \quad (15)$$

However, it has been empirically proven that a fixed KL-penalty coefficient is generally not sufficient. This motivates the need for PPO, which appends several benefits to the TRPO approach.

C.4 Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) optimizes the same objective as TRPO, including the Lagrangian-relaxation of the KL divergence, but makes two small tweaks.

C.4.1 PPO-Penalty

PPO-Penalty optimizes the same TRPO objective mentioned earlier, in 15, but with adaptive penalties. For each iteration

- solve 15 with a fixed β , then
- For the policy obtained from the previous step, compute

$$d = \hat{\mathbb{E}} [D_{\text{KL}}(\pi_{\theta_k}(\cdot | s), \pi_{\theta_{k+1}}(\cdot | s))] \quad (16)$$

If

- $d \leq d_{\text{target}}$, decrease β
- $d \geq d_{\text{target}}$, increase β

C.4.2 PPO-Clip

PPO-Clip, as the name suggests, truncates the policy mismatch. If $\hat{A}_{\pi_{\theta_k}}^R(s, a) > 0$,

$$\hat{J}_{\pi_{\theta_k}}(\pi_{\theta_{k+1}}) = \max \left\{ \frac{\pi_{\theta_{k+1}}(a|s)}{\pi_{\theta_k}(a|s)}, 1 - \epsilon \right\} \hat{A}_{\pi_{\theta_k}}^R(s, a) \quad (17)$$

If $\hat{A}_{\pi_{\theta_k}}^R(s, a) < 0$,

$$\hat{J}_{\pi_{\theta_k}}(\pi_{\theta_{k+1}}) = \max \left\{ \frac{\pi_{\theta_{k+1}}(a|s)}{\pi_{\theta_k}(a|s)}, 1 + \epsilon \right\} \hat{A}_{\pi_{\theta_k}}^R(s, a) \quad (18)$$

Hence, the new obtained surrogate is

$$\hat{J}_{\pi_{\theta_k}}^{\text{clip}}(\pi_{\theta_{k+1}}) = \mathbb{E} \left[\min \left\{ \frac{\pi_{\theta_{k+1}}(a|s)}{\pi_{\theta_k}(a|s)} \hat{A}_{\pi_{\theta_k}}^R(s, a), \text{clip} \left(\frac{\pi_{\theta_{k+1}}(a|s)}{\pi_{\theta_k}(a|s)}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{\pi_{\theta_k}}^R(s, a) \right\} \right] \quad (19)$$

Where ϵ is a small constant whose value relative to the ratio of the policies indicates some notion of dissimilarity between the policies at hand. *Generalized Advantage Estimation* (GAE) is a popular method well-suited to recurrent neural networks. For brevity, this has been left inexplicit.

C.5 CMDPS - Lagrangian Formulations

The general approach one takes to CMDPs is as follows

$$\pi_{k+1} = \arg \max_{\pi \in \Pi_{\theta}} J^R(\pi), \text{ s.t. } J^C(\pi) \leq d, \text{ and } D(\pi, \pi_k) \leq \delta \quad (20)$$

20 may be relaxed as

$$\min_{\lambda \geq 0} \max_{\theta} \left\{ J^R(\pi) - \lambda J^C(\pi) \right\} \quad (21)$$

where $\lambda \geq 0$ is the Lagrange multiplier associated with the cost constraint, the increment of which makes 21 converge into the expression 20 (without KL-constraint). The Lagrangian method proceeds in two steps

- First solve the optimization problem 21 and obtain the optimal $\theta^*(\lambda)$ for a fixed λ
- Then gradually increase the constraint violation penalty (multiplier) until satisfaction

Note - It is important noting that the KL-penalty is an algorithmic constraint and not an intrinsic part of the model itself. The region of all policies sufficiently close in the KL-distance is exactly what the *Trust Region* in TRPO stands for. In general, the first step is not complicated and fast. The multiplier updates follow

$$\min_{\lambda \geq 0} (1 - \lambda) \left\{ J^R(\pi) - \lambda J^C(\pi) \right\} \quad (22)$$

Algorithmically, the updates follow a simple procedure

$$\lambda_{k+1} = \max \left\{ \lambda_k + \alpha [J^C(\pi) - d], 0 \right\} \quad (23)$$

where α is some learning rate.

C.6 PPO-Lagrangian (PPO-Lag)

PPO-Lagrangian extends the Lagrangian formulation of CMDPs in (21) by combining it with the proximal updates of PPO. In particular, the policy update is performed using a clipped surrogate objective while incorporating a Lagrangian penalty on constraint violations.

For a fixed multiplier λ_k , the objective becomes

$$\max_{\theta_{k+1}} \mathbb{E} \left[\min \left(\frac{\pi_{\theta_{k+1}}(a|s)}{\pi_{\theta_k}(a|s)} \hat{A}_{\pi_{\theta_k}}^R(s, a), \text{clip} \left(\frac{\pi_{\theta_{k+1}}(a|s)}{\pi_{\theta_k}(a|s)}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{\pi_{\theta_k}}^R(s, a) \right) \right] - \lambda_k J^C(\pi_{\theta_{k+1}}). \quad (24)$$

Equivalently, using a cost advantage estimate $\hat{A}_{\pi_{\theta_k}}^C(s, a)$, the objective may be written as

$$\max_{\theta_{k+1}} \mathbb{E} \left[\frac{\pi_{\theta_{k+1}}(a|s)}{\pi_{\theta_k}(a|s)} \left(\hat{A}_{\pi_{\theta_k}}^R(s, a) - \lambda_k \hat{A}_{\pi_{\theta_k}}^C(s, a) \right) \right], \quad (25)$$

with clipping applied in the same manner as in (19).

The multiplier is updated according to the dual ascent step

$$\lambda_{k+1} = \max \left(\lambda_k + \alpha (J^C(\pi_{\theta_k}) - d), 0 \right), \quad (26)$$

which penalizes constraint violations and drives the policy toward feasibility.

Remark. Unlike CPO, which enforces constraints explicitly at each update via a constrained optimization problem, PPO-Lag incorporates constraints indirectly through the Lagrangian penalty. As a result, constraint satisfaction is not guaranteed at every iteration but is typically achieved asymptotically as the multiplier converges.

C.7 Constrained Policy Optimization (CPO)

The TRPO objective may be extended further to account for constraints on costs as follows

$$\pi_{k+1} = \arg \max_{\pi \in \Pi_\theta \cap \Pi_c} J^R(\pi), \quad \text{s.t. } D_{KL}(\pi_k || \pi) \leq \delta, \text{ and } J^C(\pi) \leq d_i \quad (27)$$

where Π_c is a set of policies which satisfy the constraint. In practice, however, finding such a set of permissible policies is intractable as it consists of a constraint evaluation step followed by a verification step. The same approximation used to simplify the TRPO objective apply here, for example, the use of sampled returns to compute an empirical KL-distance, etc. CPO [1] provides an alternative to this by replacing D_{KL} with a quadratic approximation (Fisher distance) with which one is able to formulate a computable surrogate for the constraints and the objective. In addition, the method allows the establishment of lower bounds on the performance and constraint violation rate as a function of hyperparameters.

Theorem 4. For any function $f : \mathcal{S} \rightarrow \mathbb{R}$ and policies π_k, π_{k+1} , define the TD-residual, it's upper bound, and the surrogate

$$\delta^f(s, a, s') := R(s, a, s') + \gamma f(s') - f(s), \quad (28)$$

$$\epsilon_{\pi_{k+1}}^f := \max_s \left| \mathbb{E}_{a \sim \pi_{k+1}, s' \sim P} [\delta^f(s, a, s')] \right|, \quad (29)$$

$$\hat{J}_{\pi_k, f}(\pi_{k+1}) := \mathbb{E}_{\tau \sim \pi_k} \left[\left(\frac{\pi_{k+1}(a|s)}{\pi_k(a|s)} - 1 \right) \delta^f(s, a, s') \right]. \quad (30)$$

Note - When $f = V^\pi$, $\mathbb{E}[\cdot]$ of δ^f is the advantage function. Lower and upper bounds may now be defined as

$$D_{\pi_k, f}^\pm(\pi_{k+1}) := \frac{\hat{J}_{\pi_k, f}(\pi_{k+1})}{1 - \gamma} \pm \frac{2\gamma\epsilon_{\pi_{k+1}}^f}{(1 - \gamma)^2} \mathbb{E}_{s \sim d_{\pi_k}} [D_{TV}(\pi_{k+1} || \pi_k)[s]], \quad (31)$$

where

$$D_{TV}(\pi_{k+1} || \pi_k)[s] = \frac{1}{2} \sum_a |\pi_{k+1}(a|s) - \pi_k(a|s)|. \quad (32)$$

Then,

$$D_{\pi_k, f}^+(\pi_{k+1}) \geq J(\pi_{k+1}) - J(\pi_k) \geq D_{\pi_k, f}^-(\pi_{k+1}). \quad (33)$$

One gets explicit bounds on the performance difference when $\delta^f = A_\pi$. For concision the expressions have been left inexplicit. The interested reader should refer to [1]

D Experimental Configurations

Experiments to comprehensively assess the performance of all algorithms on all benchmarks across multiple seeds are compute-intensive. Hence, we intentionally employ different algorithm sets across the two benchmarks to reflect the distinct structural properties of the environments.

The SafeTreasureGoal environment (Figure 5) introduces temporally extended dependencies and logical task specifications (key-door-goal sequencing) in a discrete setting; we feel that such a structure is not adequately

Random Layout 1

			cliff	cliff	cliff
T4	T3		start	T1	
			wall		K
		D			
	T2		wall		G

Figure 5: Visual representation of the SafeTreasureGoal environment. The agent must satisfy the temporal logic of collecting the key, opening the door, and reaching the goal while avoiding cliffs.

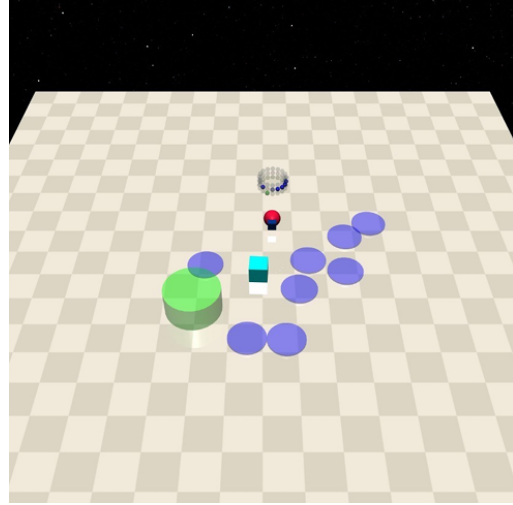


Figure 6: Visual representation of the SafetyPointGoal1-v0 environment. The point-mass agent navigates to the goal while avoiding dynamic hazard zones.

captured by standard constrained MDP formulations. Consequently, we evaluate LTL-PPO alongside PPO and PPO-LAG to assess the benefit of incorporating formal task structure into policy optimization.

In contrast, SafetyPointGoal1-v0 (Figure 6) is a standard constrained MDP benchmark with Markovian dynamics and explicit cost signals but no temporal logic requirements. Therefore, we evaluate PPO, PPO-LAG, and CPO, which are specifically designed for constraint satisfaction in such settings. This separation ensures that each algorithm is evaluated in the regime for which it is designed, enabling a fair and meaningful comparison of their respective strengths.

Training Configuration - SafeTreasureGoal (Random with early goal penalty)

Seeds: $S = \{1, 2, 3, 4, 5\}$

$$\text{Main} \quad \left\{ \begin{array}{l} \text{total_steps} = 5 \times 10^5 \\ \text{rollout_steps} = 1024 \\ \text{update_epochs} = 8 \\ \text{minibatch_size} = 256 \end{array} \right.$$

$$\text{LTL-PPO} \quad \left\{ \begin{array}{l} \gamma = 0.99, \quad \lambda = 0.95 \\ \epsilon_{\text{clip}} = 0.20 \\ c_v = 0.50, \quad c_{\text{ent}} = 0.04 \\ \alpha = 3 \times 10^{-4} \\ r_{\text{progress}} = 1.91 \\ c_{\text{cliff}} = 0.0 \\ \text{terminate_on_cliff} = \text{False} \end{array} \right.$$

$$\text{PPO} \quad \left\{ \begin{array}{l} \gamma = 0.99, \quad \lambda = 0.95 \\ \epsilon_{\text{clip}} = 0.20 \\ c_v = 0.50, \quad c_{\text{ent}} = 0.04 \\ \alpha = 3 \times 10^{-4} \end{array} \right.$$

$$\text{PPO-Lag} \quad \left\{ \begin{array}{l} \gamma = 0.99, \quad \gamma_c = 0.99 \\ \lambda = 0.95 \\ \epsilon_{\text{clip}} = 0.20 \\ c_v = 0.50, \quad c_v^{(c)} = 0.50 \\ c_{\text{ent}} = 0.04 \\ \alpha = 3 \times 10^{-4} \\ \eta_{\lambda} = 0.01 \\ d = 1.0 \end{array} \right.$$

Training Configuration - SafeTreasureGoal (Random without early goal penalty)

Seeds: $S = \{1, 2, 3, 4, 5\}$

$$\begin{aligned}
 \text{Main} & \begin{cases} \text{total_steps} = 1 \times 10^6 \\ \text{rollout_steps} = 2048 \\ \text{update_epochs} = 8 \\ \text{minibatch_size} = 256 \end{cases} \\
 \text{LTL-PPO} & \begin{cases} \gamma = 0.99, \quad \lambda = 0.95 \\ \epsilon_{\text{clip}} = 0.20 \\ c_v = 0.50, \quad c_{\text{ent}} = 0.04 \\ \alpha = 3 \times 10^{-4} \\ r_{\text{progress}} = 1.91 \\ c_{\text{cliff}} = 0.0 \\ \text{terminate_on_cliff} = \text{False} \end{cases} \\
 \text{PPO} & \begin{cases} \gamma = 0.99, \quad \lambda = 0.95 \\ \epsilon_{\text{clip}} = 0.20 \\ c_v = 0.50, \quad c_{\text{ent}} = 0.04 \\ \alpha = 3 \times 10^{-4} \end{cases} \\
 \text{PPO-Lag} & \begin{cases} \gamma = 0.99, \quad \gamma_c = 0.99 \\ \lambda = 0.95 \\ \epsilon_{\text{clip}} = 0.20 \\ c_v = 0.50, \quad c_v^{(c)} = 0.50 \\ c_{\text{ent}} = 0.04 \\ \alpha = 3 \times 10^{-4} \\ \eta_\lambda = 0.01 \\ d = 1.0 \end{cases}
 \end{aligned}$$

Training Configuration - SafetyPointGoal1-v0 (Continuous Control)

Seeds: $S = \{0, 1, 2\}$

$$\begin{aligned}
 \text{Main (OmniSafe Framework)} & \begin{cases} \text{total_steps} = 1,024,000 \\ \text{steps_per_epoch} = 2048 \\ \text{update_iters} = 10 \\ \text{vector_env_nums} = 1 \\ \text{cost_limit}(d) = 25.0 \end{cases} \\
 \text{PPO \& CPO (Defaults)} & \begin{cases} \gamma = 0.99 \\ \lambda_{\text{GAE}} = 0.97 \\ \epsilon_{\text{clip}} = 0.20 \quad (\text{PPO only}) \end{cases} \\
 \text{PPO-Lag Specific} & \begin{cases} \lambda_{\text{init}} = 0.0 \\ \text{Penalty learning rate } (\alpha) = \text{OmniSafe Default} \end{cases}
 \end{aligned}$$

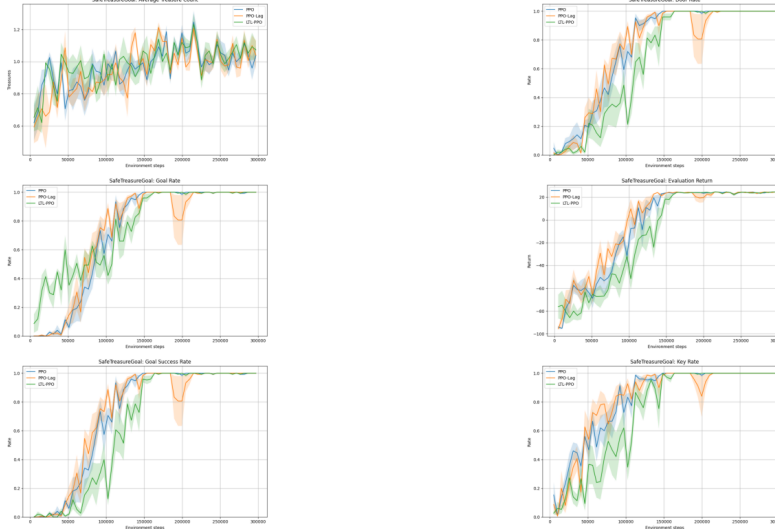


Figure 7: Algorithm sweep on the SafeTreasureGoal benchmark (fixed) displaying performance metrics across 5 seeds. Success rates imply a complete K>D>G sequence.

Figure 7 depicts the effect of fixed starting tiles on the performance.

Compute Resources and Infrastructure

All experiments were executed on a High-Performance Computing (HPC) cluster utilizing Lambda GPU instances. Hardware acceleration was provided by NVIDIA A10 Tensor Core GPUs. For the continuous control environments (SafetyPointGoal1-v0), training a single algorithm across 3 independent random seeds for 1.024×10^6 timesteps required approximately 4 hours of total wall-clock time. Discrete environment evaluations (SafeTreasureGoal) were conducted under identical hardware configurations, ensuring consistent and equitable computational availability across all baseline comparisons.

AIS extension. Now suppose $\hat{z}_t = \hat{\sigma}(H_t)$ is not an exact information state, but an (ε, δ) -approximate information state. That is,

$$|\mathbb{E}[r(S_t, a) \mid H_t, A_t = a] - \hat{r}(\hat{z}_t, a)| \leq \varepsilon,$$

and

$$d_{\mathcal{F}}\left(\mathbb{P}(\hat{Z}_{t+1} \in \cdot \mid H_t, A_t = a), \hat{P}(\cdot \mid \hat{z}_t, a)\right) \leq \delta.$$

Let $\hat{J}^R$ denote the return in the AIS-induced surrogate MDP

$$\hat{M}_Z = (\hat{\mathcal{Z}}, \mathcal{A}, \hat{P}, \hat{r}, \gamma).$$

Assume the value functions are uniformly bounded by

$$\|V^\pi\|_\infty \leq V_{\max}.$$

Then, for any fixed policy π ,

$$|J^R(\pi) - \hat{J}^R(\pi)| \leq \frac{\varepsilon}{1 - \gamma} + \frac{\gamma V_{\max}}{1 - \gamma} \delta.$$

Consequently, applying the CPI bound inside the AIS-induced MDP gives

$$J^R(\pi_{\text{new}}) \geq \hat{J}_{Z, \pi_{\text{old}}}^R(\pi_{\text{new}}) - \frac{2\hat{\varepsilon}_Z \gamma}{(1 - \gamma)^2} \alpha^2 - 2 \left(\frac{\varepsilon}{1 - \gamma} + \frac{\gamma V_{\max}}{1 - \gamma} \delta \right). \quad (34)$$